

Detecting Adversarial Attacks in Image Sequences with Conformal Test Martingales

Genghua Dong

Ziyun Li

Henrik Boström

KTH Royal Institute of Technology

GENGHUA@KTH.SE

ZIYLI@KTH.SE

BOSTROMH@KTH.SE

Editor: Ernst Ahlberg, Ulf Johansson, Henrik Boström, Alberto Carlevaro, Johan Hallberg Szabadváry and Lars Carlsson

Abstract

Deep neural networks used for image classification remain vulnerable to adversarial examples (AEs), especially under white-box attacks where the attacker can exploit information about the target model. Detecting whether a model is under attack is therefore an important step toward mitigating adversarial manipulation. Most existing AE detectors focus on instance-level or batch-level decisions, i.e., determining whether a single image or a fixed batch contains AEs. In this paper, we study a sequence-level detection setting using conformal test martingales. We formulate several nonconformity scores based on image and target-model quantities, convert them into conformal p-values, and monitor the resulting sequences with conformal test martingale algorithms. The resulting detection procedure is evaluated on ImageNet-100 image sequences with a pretrained ResNet-50 target model and three representative white-box attacks. The experiments show that conformal test martingales can detect attacks even when the monitoring segment contains a relatively small number of adversarial examples, but that detection performance depends strongly on both the nonconformity score and the martingale algorithm. Empirically, no single nonconformity score and algorithm pair works well for all attacks. These findings suggest that applying conformal test martingales to detect attacks may benefit from considering multiple image and target-model quantities and designing nonconformity scores according to the observed attack-induced changes.

Keywords: Conformal test martingale, adversarial examples, white-box attacks

1. Introduction

Deep neural networks are widely used in image classification, but they remain vulnerable to adversarial examples (AEs): inputs modified by small, carefully constructed perturbations that can change model predictions while remaining visually inconspicuous to humans. Early work showed that such perturbations can be generated efficiently by using gradient information, and later work developed stronger optimization-based attacks that became standard tools for evaluating adversarial robustness (Goodfellow et al., 2014; Madry et al., 2017; Carlini and Wagner, 2017b).

Techniques for generating attacks by AEs are commonly divided into two categories: white-box and black-box attacks. In a white-box setting, the attacker has access to the target model, including information such as the model architecture, parameters, loss function, or gradients. In a black-box setting, the attacker does not have full access to the target

model and must instead rely on model outputs, queries, or surrogate models (Wiyatno et al., 2019).

Since white-box attacks can directly use information about the target model during optimization, they provide a strong and direct stress test for attack detection methods (Wang et al., 2022; Ebrahimi et al., 2018; Carlini and Wagner, 2017a). There are several typical white-box attack techniques. Projected gradient descent (PGD) is an iterative gradient-based attack that repeatedly updates the input in the direction that increases the target model’s loss, followed by projection back to the allowed perturbation set (Madry et al., 2017). The Carlini–Wagner (CW) attack is an optimization-based attack that searches for adversarial examples under explicit distance-based objectives and has been widely used to test whether defenses remain robust under stronger attack objectives (Carlini and Wagner, 2017b). Adversarial patch (AP) attacks optimize a localized patch that can be placed on an image and can remain effective under transformations (Brown et al., 2017). These three attacks represent different perturbation methods: iterative gradient perturbations, optimization-based perturbations, and localized patch perturbations.

Therefore, detecting whether a model is under attack is an important first step towards mitigating or avoiding adversarial attempts to change the outcome of the model. Many existing detection methods are designed to make an instance-level decision. For example, detector subnetworks can be trained on hidden-layer representations to distinguish clean and adversarial inputs (Metzen et al., 2017); feature squeezing detects AEs by comparing predictions on the original image and transformed versions of the same image (Xu et al., 2017); another approach, BEYOND, detects AEs by examining the relation between an input image and its augmented neighbors, showing that non-attacked (clean) samples preserve stronger representation similarity and label consistency under augmentations, while AEs exhibit weaker consistency with their neighbors (He et al., 2022); and other methods use density estimates, uncertainty, reconstruction error, or local intrinsic dimensionality in feature space (Feinman et al., 2017; Meng and Chen, 2017; Ma et al., 2018). Batch-level detection has also been studied, for example by using geometric statistics such as density and coverage to decide whether a fixed batch contains adversarial samples (Venkatesh and Steinbach, 2022). These methods are useful, but they mainly address whether a single image, or a fixed batch of images, appears adversarial.

In contrast, our work addresses the detection of AEs on an image sequence. This setting consists of a clean reference segment followed by a monitoring segment. The goal is not to classify every individual image as clean or adversarial, but to detect whether the monitoring sequence shows evidence of attack-induced distribution shift. If AEs are mixed into the monitoring segment, the distribution of image-derived quantities may change. These quantities may include image statistics, model embeddings, logits, gradients, or other score functions computed from the images.

Conformal test martingales (CTMs) provide a natural framework for this sequence-level detection problem. A CTM first maps each example to a nonconformity score (NCS), converts the resulting score sequence into conformal p-values, and then updates a test martingale from the p-values. Under the null hypothesis that the sequence is exchangeable, the martingale has a threshold-based false-alarm guarantee.

We extend a previous study that applied CTMs to white-box attack detection in image sequences (Dong et al., 2025). While the CTM framework itself is well established, its

detection power on attack-induced sequences depends critically on the choice of NCS: the martingale can only accumulate evidence of drift if the NCS first exposes that drift in its score distribution. The research question we investigate is therefore: *how to define an NCS based on the input image and the target model to allow detecting attack-induced distribution shift with a CTM?* Concretely, we construct several NCSs from image and target-model quantities, including frequency domain statistics, model logits, and embedding consistency between an image and its benign views. With a suitable choice of CTM algorithm, some of these NCSs allow detecting shifts caused by all three attack types considered in this work when AEs constitute about 20% to 30% of the monitoring sequence, while other NCSs mainly detect PGD and AP at similar AE proportions.

The main contributions of this work are:

- Several NCSs for CTM detection of white-box attacks in image sequences are proposed. These NCSs use benign views, frequency domain statistics, embeddings, and logits.
- The proposed NCSs are empirically evaluated using three representative CTM algorithms and selected variants. The results indicate that successful detection depends on the type of distribution shift reflected in the NCS sequence and on the martingale algorithm used to process the resulting p-values.

In the next section, we describe the application of conformal test martingales to detecting attacks, including the proposed nonconformity functions. In Section 3, we describe the experimental setup for the empirical investigation and present the results in Section 4. Finally, we summarize the main findings and outline directions for future work in Section 5.

2. Methodology

2.1. CTMs for Detecting Attack-Induced Distribution Shift

We observe a sequence

$$z_1, z_2, \dots,$$

where each $z_i \in \mathcal{Z}$ for some example space $\mathcal{Z}$ and where an initial part of the sequence is assumed to be clean (non-attacked). Let q denote the last index of this initial clean part. For NCSs requiring reference information, we use the first r images, where $r \leq q$, as reference images and define

$$\mathcal{R} = \{z_1, \dots, z_r\}.$$

The reference images are used only to define the corresponding NCSs and are not used for computing conformal p-values or martingales. The remaining clean images, $z_{r+1}, \dots, z_q$, form the initial clean segment for p-value and martingale computation. The subsequent images, $z_{q+1}, \dots$, form the monitoring segment, which may contain AEs.

The NCSs considered in this work may contain one or more scalar components. Conformal p-values and martingales are computed separately for each component before the resulting component martingales are combined. We therefore first describe the CTM construction for a generic scalar component; the aggregation of multiple component martingales is specified in Section 2.3.

Let

$$A : \mathcal{Z} \rightarrow \mathbb{R}$$

denote a scalar nonconformity function. The function A , constructed using the reference images when needed, maps each example z_n , $n \geq r + 1$, to an NCS, denoted by

$$\alpha_n = A(z_n).$$

By convention, the NCSs in this work are constructed such that a larger value of α_n indicates that z_n is less conforming according to A . From the sequence of NCSs, $\alpha_{r+1}, \alpha_{r+2}, \dots$, we compute smoothed conformal p-values. In the semi-online mode, the p-value for the n -th score, $n \geq r + 1$, is

$$p_n = \frac{|\{i \in \{r + 1, \dots, n\} : \alpha_i > \alpha_n\}| + \tau_n |\{i \in \{r + 1, \dots, n\} : \alpha_i = \alpha_n\}|}{n - r}, \quad \tau_n \sim \text{Unif}(0, 1).$$

After the reference images have been fixed, under the exchangeability assumption for the sequence on which p-values are computed, these p-values are uniformly distributed on $[0, 1]$ and can be used to construct a conformal test martingale (Vovk, 2021; Vovk et al., 2022; Boström, 2026). A conformal test martingale is initialized with $s_r = 1$ and then computed as

$$s_n = \prod_{i=r+1}^n f_i(p_i),$$

where each $f_i : [0, 1] \rightarrow [0, \infty]$ is a betting function satisfying

$$\int_0^1 f_i(u) du = 1.$$

In practice, the values of s_n are obtained from a CTM algorithm, such as Simple Jumper, Sleeper/Stayer, or Sleeper/Drifter (Vovk et al., 2022).

If the sequence on which p-values are computed is exchangeable, the process $(s_n)_{n \geq r}$ with $s_r = 1$ is a non-negative martingale with $\mathbb{E}[s_n] = 1$ for all $n \geq r$, and Ville's inequality gives

$$\mathbb{P}(\exists n \geq r + 1 : s_n \geq c) \leq \frac{1}{c}.$$

The conformal test martingale can therefore be used for testing the exchangeability assumption: a violation is signaled when s_n reaches or exceeds a predefined threshold c . Choosing c controls the probability of a false alarm under the null hypothesis of exchangeability.

In our image sequence setting, the construction of NCSs is one important part of the detection procedure. The resulting score distribution should remain stable on clean sequences, while reflecting the distribution shift introduced by AEs.

2.2. Candidate Nonconformity Scores

We consider three groups of NCSs in this paper. The first group is constructed from logits of the target model. The second group is constructed from embeddings of the sequence images and their distances to embeddings of clean reference images. The third group uses additional information derived from the input image, either by comparing an image with a benign transformed view of itself in the embedding space, or by computing frequency statistics directly from the input tensor.

2.2.1. LOGIT-BASED NCSs

We consider two types of logit-based NCSs. Let $\ell_n \in \mathbb{R}^{d_\ell}$ denote the logit vector of image x_n . The first type is the raw logit NCS, which takes each raw logit as one scalar NCS component. The other type is the logit statistic NCS, which consists of several NCS components, each using a scalar statistic of the logit vector.

Raw logit NCS. The raw logit NCS takes each logit as one scalar component:

$$A_j^{\text{rawlogit}}(z_n) = \ell_{n,j}, \quad j = 1, \dots, d_\ell.$$

The resulting components are used to test whether the distribution of logits in the output vector shifts when AEs are present in the image sequence.

Logit statistic NCS. The logit statistic NCS consists of three scalar components computed from the logit vector. Let $\ell_{n,(1)}$ and $\ell_{n,(2)}$ denote the largest and second largest values in ℓ_n , respectively. We define

$$A^{\text{std}}(z_n) = \text{std}(\ell_n), \quad A^{\text{margin},-}(z_n) = \ell_{n,(2)} - \ell_{n,(1)}, \quad A^{\text{margin},+}(z_n) = \ell_{n,(1)} - \ell_{n,(2)}.$$

Although $A^{\text{margin},+} = -A^{\text{margin},-}$ as scalars, each component is converted into its own p-value sequence and then into its own martingale before being averaged. Because standard CTM betting functions concentrate on small p-values to detect large-NCS deviations, including both signed margins allows the averaged martingale to capture an attack-induced top-1/top-2 gap change regardless of whether the gap widens (e.g., a misclassified AE with high confidence) or narrows (e.g., an AE that produces near-tied top-2 logits). The std component complements the margins by reflecting changes in the overall dispersion of ℓ_n . Together, these three components cover both location- and scale-type drift induced by the considered attacks.

2.2.2. EMBEDDING NCSs

The embedding NCSs are constructed from hidden representations extracted from the target model. Let $h(x_n)$ denote the selected embedding of image x_n , and define the L2 normalized embedding

$$\bar{h}(x_n) = \frac{\text{vec}(h(x_n))}{\|\text{vec}(h(x_n))\|_2}.$$

For each reference image x_i , $i \in \mathcal{R}$, we define one scalar NCS component:

$$A_i^{\text{emb}}(z_n) = 1 - \bar{h}(x_n)^\top \bar{h}(x_i).$$

Each component measures the cosine distance from the current image to one fixed clean reference image. Larger values indicate that the current embedding is farther from that reference embedding. This yields one scalar component per clean reference image.

2.2.3. INPUT-DERIVED NCSs

The third group contains NCSs that introduce information beyond the model outputs of the original image. It includes a benign-view NCS, which compares an image with a transformed view of itself in the embedding space, and an FFT NCS, which is computed directly from image frequency statistics.

Benign-view NCS. For an image tensor x_n , let $v(x_n)$ denote a fixed benign transformation of the same tensor. Let $h(\cdot)$ denote the selected model embedding, and let $\bar{h}(\cdot)$ denote the corresponding L2 normalized embedding. The benign-view NCS is defined as

$$A^{\text{ben}}(z_n) = 1 - \bar{h}(x_n)^\top \bar{h}(v(x_n)).$$

A larger value indicates weaker consistency between the original image and its benign view in the embedding space. This score differs from the embedding NCS above because the comparison is made within the same image identity, rather than between the current image and the clean reference images.

Tensor FFT NCS. Frequency-domain information has been shown to be relevant for both CNN behaviour and adversarial examples: CNNs may exploit high-frequency image components that are weakly aligned with human perception, and adversarial perturbations have been analysed from a Fourier perspective (Yin et al., 2019; Wang et al., 2020); Fourier-domain analysis of input images and feature maps has also been used for adversarial-image detection (Harder et al., 2021). Motivated by these observations, we define an FFT NCS that is computed directly from the input tensor and does not rely on model outputs or embeddings. For each image tensor x_n , we first compute its power spectrum using the fast Fourier transform (FFT), from which a small set of frequency statistics is derived. Let $Q_j(x_n)$ denote the j -th statistic.

The statistic is standardized relative to the clean reference segment. Specifically, we define the robust location and scale as

$$\mu_j = \text{median}\{Q_j(x_i) : i \in \mathcal{R}\}, \quad s_j = \text{MAD}\{Q_j(x_i) : i \in \mathcal{R}\},$$

where

$$\text{MAD}\{Q_j(x_i) : i \in \mathcal{R}\} = \text{median}\{|Q_j(x_i) - \mu_j| : i \in \mathcal{R}\}.$$

The standardized statistic is then

$$Z_j(x_n) = \frac{Q_j(x_n) - \mu_j}{s_j}.$$

If the MAD is degenerate, i.e., $s_j = 0$, the corresponding statistic is constant on the reference segment and is dropped from the NCS rather than standardized; in our experiments this case did not occur for any of the FFT statistics on the reference segments used. For each statistic, we consider both upper and lower tail components:

$$A_{j,+}^{\text{fft}}(z_n) = \max\{Z_j(x_n), 0\}, \quad A_{j,-}^{\text{fft}}(z_n) = \max\{-Z_j(x_n), 0\}.$$

These components indicate whether a frequency statistic is unusually large or unusually small relative to the clean reference segment.

2.3. Martingale Computation and Signaling

For each scalar NCS component, we compute a sequence of conformal p-values as described in Section 2.1 and then obtain a CTM with a selected martingale algorithm. For NCSs with a single component, this martingale is employed directly for signaling. Other NCSs, such

Algorithm 1: CTM detection pipeline

Input : X_{clean} clean image pool
 X_{adv} adversarial image pools
 $\mathcal{S}$ set of NCSs
 $\mathcal{A}, \mathcal{G}, \mathcal{P}$ attack methods, martingale algorithms, attack rates
 c detection threshold

Output: detection outcomes and first signaling indices

- 1 Generate reference segments, initial segments, monitoring segments, and matched clean-control sequences
- 2 **forall** $(a, \rho) \in \mathcal{A} \times \mathcal{P}$ **do**
- 3 $Z_{\text{atk}} \leftarrow$ attack sequence from (a, ρ)
- 4 $Z_{\text{clean}} \leftarrow$ matched clean-control sequence
- 5 **forall** NCS $S \in \mathcal{S}$ **do**
- 6 Construct S with the reference segment when needed
- 7 $\mathbf{A}_{\text{atk}} \leftarrow S(Z_{\text{atk}})$, $\mathbf{A}_{\text{clean}} \leftarrow S(Z_{\text{clean}})$
- 8 **forall** algorithm $g \in \mathcal{G}$ **do**
- 9 $M_{\text{atk}} \leftarrow \text{CTM}(\mathbf{A}_{\text{atk}}, g)$, $M_{\text{clean}} \leftarrow \text{CTM}(\mathbf{A}_{\text{clean}}, g)$
- 10 **if** S has multiple scalar components **then**
- 11 $M_{\text{atk}} \leftarrow \text{Average}(M_{\text{atk}})$, $M_{\text{clean}} \leftarrow \text{Average}(M_{\text{clean}})$
- 12 $n_{\text{atk}} \leftarrow \inf\{n : M_{\text{atk}}(n) \geq c\}$, $n_{\text{clean}} \leftarrow \inf\{n : M_{\text{clean}}(n) \geq c\}$
- 13 **if** n_{atk} exists **and** n_{clean} does not exist **then**
- 14 **record** successful detection at index n_{atk}
- 15 **else**
- 16 **record** no successful detection

as the logit-based NCSs, contain multiple scalar components. For such NCSs, we obtain one martingale for each component. We then follow the uniformly weighted martingale construction of [Boström \(2025\)](#) and average the component martingales pointwise. The resulting averaged martingale is employed for signaling.

A violation of exchangeability is signaled when the resulting martingale reaches or exceeds a predefined threshold c .

The complete detection and evaluation pipeline is summarized in Algorithm 1. For each attack sequence, we also construct a matched clean-control sequence. The matched clean-control sequence is not required for the type-I error control provided by Ville’s inequality. We include it to adopt a more conservative empirical detection criterion. A detection is counted as successful only when the martingale for the attacked sequence reaches or exceeds the threshold while the martingale for its matched clean-control sequence remains below the threshold.

3. Experimental Setup

3.1. Dataset and Attacks

Dataset and target model. We conduct experiments on ImageNet-100 (Tian et al., 2020), a 100-class subset of ImageNet-1K (Russakovsky et al., 2015). ImageNet-100 has 126 689 images in the training set and 5 000 images in the validation set. The target model is a ResNet-50 model (He et al., 2016) pretrained on ImageNet-1K and implemented in PyTorch (Paszke et al., 2019). We directly employ the pretrained model without replacing the classification head or fine-tuning it on ImageNet-100. Hence, the model still outputs logits over the original 1000 ImageNet classes, although the images used to generate the sequences are restricted to the 100 classes in ImageNet-100.

For the NCSs that rely on embeddings, including the embedding-based NCSs and benign-view NCSs, we extract the global average pooled stage features produced by the same ResNet-50. We denote the feature extracted from stage ℓ by

$$E_{\ell, \text{gap}}(x), \quad \ell \in \{1, 2, 3, 4\}.$$

Unless otherwise stated, all NCSs that rely on embeddings are computed from $E_{4, \text{gap}}$, the global average pooled feature extracted from the fourth stage.

Adversarial images generation. We generate AEs using three white-box attack methods: PGD, CW, and adversarial patch (AP). All adversarial images are generated using the Adversarial Robustness Toolbox (ART) (Nicolae et al., 2018). We randomly select the same 4000 images from the ImageNet-100 validation set for all three attack methods. For each attack method, one AE is generated for each selected clean image, resulting in 4000 AEs per attack. The attacks are untargeted and are optimized against the ImageNet-100 logit space of the ResNet-50. Additionally, we first resize the original ImageNet-100 images to 224×224 , and then generate AEs from these resized images. This keeps the clean images and AEs at the same spatial size, avoiding distribution drifts caused by size differences rather than by the attacks themselves.

For PGD (Madry et al., 2017), the perturbation budget is $\epsilon = 0.03$, the step size is $\epsilon_{\text{step}} = 0.01$, and the attack is run for 7 iterations with no random initialization.

For CW (Carlini and Wagner, 2017b), the confidence parameter is 0, the learning rate is 0.01, the number of binary search steps is 5, the number of optimization iterations is 20, and the initial constant is 5.0.

For AP (Brown et al., 2017), the maximum rotation is 22.5° , the scale range is $[0.1, 1.0]$, the distortion scale is 0, the learning rate is 5.0, the number of iterations is 500, the patch type is circular, and the optimizer is Adam (Kingma and Ba, 2014). The patch is trained on 2000 images sampled randomly from the ImageNet-100 training set and then applied to clean images from the ImageNet-100 validation set with application scale 0.4. The patch shape is fixed by the attack-generation configuration and kept unchanged across the detection experiments.

The attack parameters are chosen to obtain reproducible white-box adversarial images, rather than exhaustive worst-case robustness evaluations. The PGD perturbation budget $\epsilon = 0.03$ and step size $\epsilon_{\text{step}} = 0.01$ are close to the perturbation budget $8/255$ and step size $2/255$ adopted in PGD robustness studies with 10 iterations (Ding et al., 2019; Gupta et al.,

2020). Our 7-step setting without random initialization is therefore standard in perturbation scale but conservative in iteration count. The CW configuration follows the standard CW setup (Carlini and Wagner, 2017b), with a relatively large initial constant but a moderate number of optimization iterations. The AP transformation and optimization settings are consistent with the adversarial patch studies (Brown et al., 2017; Braunegg et al., 2020). Although the AP application scale 0.4 corresponds to a relatively large localized patch, AP produces the smallest accuracy drop among these attacks. The accuracy drops for all three attacks are reported in the Results section.

Sequence generation and attack rate. Each detection experiment generates a sequence with a clean reference segment of length $r = 400$, followed by an initial clean segment of length $q - r = 400$ and a monitoring segment of length $N_{\text{mon}} = 1600$. The clean reference segment is used only to define the corresponding NCSs and is not used for computing conformal p-values or martingales. We ensure that the clean reference segment and the initial clean segment contain the same number of images from every class. Specifically, each clean segment contains 400 images over the 100 ImageNet-100 classes, corresponding to $400/100 = 4$ images per class in the segment, and the images are randomly sampled from the validation set of that class without overlap between the two segments.

We randomly sample images to generate monitoring segments from the clean validation dataset for which adversarial counterparts are available, also ensuring that none of them overlap with the two clean segments. To simulate an attack in the monitoring segment, we denote the attack rate by $\rho_{\text{atk}} \in [0, 1]$. For a given attack method and attack rate, each position j in the monitoring segment is assigned an independent attack indicator

$$B_j \sim \text{Bernoulli}(\rho_{\text{atk}}).$$

If $B_j = 1$, the clean image at this position is replaced by its adversarial counterpart; if $B_j = 0$, the clean image is retained. Thus, the number of AEs inserted into the monitoring segment is

$$N_{\text{AE}}(\rho_{\text{atk}}) = \sum_{j=1}^{N_{\text{mon}}} B_j,$$

with expectation $\mathbb{E}[N_{\text{AE}}(\rho_{\text{atk}})] = \rho_{\text{atk}} N_{\text{mon}}$. In our experiments, ρ_{atk} is varied from 0.1 to 0.9 with step 0.1.

For each experimental run, we also generate a matched clean-control sequence, which contains the same clean segments and the same monitoring segment before any adversarial replacements are made. Therefore, the clean-control sequence represents the no-attack condition and is used as a false-alarm check.

To improve the reliability of the results, we repeat the sequence generation procedure 10 times. After a sequence is sampled, it is kept fixed when scanning attack methods, attack rates, and martingale algorithms for that NCS.

3.2. Nonconformity Score Configurations

This subsection specifies the NCS configurations used in the experiments. Clean images and AEs are represented as 224×224 tensors with values in $[0, 1]$. Before being passed to

the model, these tensors are normalized by the ImageNet mean and standard deviation. No additional resize or center crop is applied.

Logit-based NCSs. For all logit-based NCSs, we use the 100 logits corresponding to the ImageNet-100 classes from the 1000 logits output by ResNet-50. These selected logits form the logit vector $\ell_n \in \mathbb{R}^{100}$ for image x_n . The raw logit NCS uses each logit of ℓ_n as one NCS component. The components in logit statistic NCS are computed from the same ℓ_n .

Embedding-based NCS. For the embedding-based NCS, we use the $E_{4,\text{gap}}(x)$ embedding of ResNet-50, i.e., the globally averaged output of `model.layer4`. Since the reference segment contains $r = 400$ images, this NCS contains 400 components.

Input-derived NCSs. We evaluate two input-derived NCSs: a benign-view NCS and a tensor FFT NCS.

For the benign-view NCS, we use the same $E_{4,\text{gap}}(x)$ embedding as the embedding-based NCS. Given an input tensor x_n , we generate a benign transformed view $v(x_n)$ by applying an affine scale transformation with scale 0.84 directly in tensor space. The scale factor 0.84 is chosen so that the central region preserved by the transformation roughly corresponds to a 188×188 center-crop of the 224×224 input, a perturbation magnitude that leaves the predicted class largely unchanged for clean images but interacts strongly with attack-induced patterns. We treat this scale as a fixed design choice in the present study. We then compute the NCS according to the definition in the methodology.

For the tensor FFT NCS, we compute frequency statistics directly from the unnormalized input tensor in $[0, 1]$. The model is not used for this NCS. The statistics summarize the relative amount of high-frequency power, the spatial concentration of high-frequency energy, and the total high-frequency energy. Each statistic is robustly standardized, as described in Section 2.2.3, using the clean reference segment and then split into an upper-tail and a lower-tail component. This gives six FFT components in total.

3.3. CTM Computation and Signaling

For each selected scalar NCS component, and for each attack method and attack rate, the score sequence is converted into conformal p-values and then monitored by a CTM. Unless otherwise stated, p-values are computed in the semi-online mode, where each score is compared with the score history observed so far. For NCSs with multiple scalar components, we compute one CTM for each component and then form a uniformly weighted martingale by averaging the component martingales pointwise, following Boström (2025).

The CTMs are computed with the `crepes` package (Boström, 2024). We consider the three original CTM algorithms Simple Jumper, Sleeper/Stayer, and Sleeper/Drifter, together with three variants obtained by modifying their betting functions: Simple Jumper with step betting, Sleeper/Stayer with epsilon betting, and Sleeper/Drifter with epsilon betting. These algorithms follow the CTM framework investigated by Boström (2026). They share the same false-alarm control under the null hypothesis of exchangeability, while their detection efficiency may differ across types of distribution drift.

In our experiments, the detection threshold is fixed to $c = 100$, and a violation of exchangeability is signaled when the martingale s_n first satisfies $s_n \geq c$. For each attack method, attack rate, NCS, and martingale algorithm, we obtain 10 detection outcomes,

Table 1: Attack-induced accuracy drop on the randomly selected 4000 ImageNet-100 validation images. The table reports the model accuracy on clean images and on the corresponding AEs.

Attack	Clean accuracy	AE accuracy	Accuracy drop
PGD	90.575%	2.100%	88.475%
CW	90.575%	0.100%	90.475%
AP	90.575%	52.850%	37.725%

with one outcome obtained from each independently sampled sequence. A sequence is counted as successfully detected if the attack-sequence martingale reaches $c = 100$, while the corresponding matched clean-control sequence does not.

To assess whether the detection results are consistent across independently generated sequences with different sampled images and adversarial positions, we evaluate each experimental configuration on ten sequences. We then apply the same stringent 10/10 criterion to all NCS and CTM-algorithm configurations, reporting the lowest attack rate at which all ten sequences yield valid detections, or “None” if no tested rate does.

4. Experimental Results

4.1. Attack-Induced Accuracy Drop

Before evaluating our CTM detection pipeline described in Algorithm 1, we first quantify how strongly the generated AEs affect the target model on the ImageNet-100 classification task. This step is important because weak attacks may fail to induce a detectable distribution shift, regardless of the NCS used to compute the martingale.

Table 1 reports the model accuracy on clean images and on the corresponding AEs generated from them. We refer to these two quantities as clean accuracy and AE accuracy, respectively. The model achieves a clean accuracy of 90.575%. PGD and CW are strong attacks in this setting: PGD reduces the accuracy to 2.100%, whereas CW reduces it to 0.100%. AP is weaker than PGD and CW, but it still substantially changes the model behavior, reducing the accuracy to 52.850%.

Figure 1 shows the measured monitoring segment accuracy as the attack rate ρ_{atk} is varied from 0.1 to 0.9. The accuracy decreases approximately linearly as the attack rate increases.

4.2. CTM Detection Results

Table 2 reports the lowest attack rates for successful CTM detection. Each cell gives the lowest attack rate at which 10/10 valid detections are achieved and the selected CTM algorithm. A valid detection means that the martingale of the monitoring segment reaches $c = 100$, while the matched clean-control martingale does not. If no algorithm reaches 10/10 valid detections at any tested attack rate ρ_{atk} , the cell is reported as “None”. Detailed algorithm-level results are reported in Appendix A, Table 3.

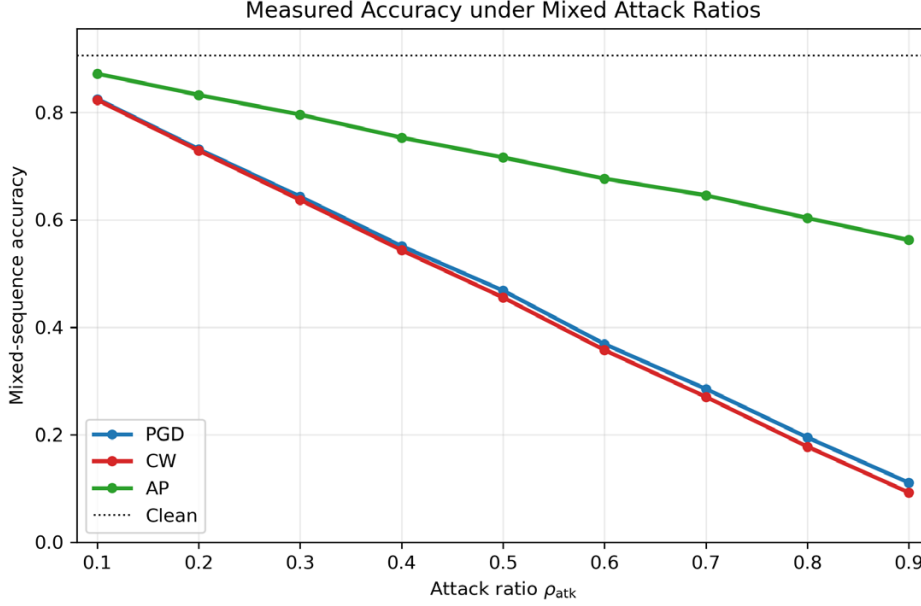


Figure 1: Measured monitoring segment accuracy under mixed attack rates. The dotted horizontal line indicates the accuracy on the matched clean-control monitoring segment. PGD and CW induce strong accuracy degradation, while AP induces a weaker but still clear shift.

Table 2: Lowest attack rate for successful CTM detection. Each cell reports the lowest attack rate at which 10/10 valid detections are achieved, together with the selected CTM algorithm abbreviation. Algorithm abbreviations are: SS = Sleeper/Stayer, SS_ϵ = Sleeper/Stayer Epsilon, SD = Sleeper/Drifter, and SD_ϵ = Sleeper/Drifter Epsilon.

NCS	PGD	CW	AP
Benign-view NCS	0.2 SD	0.2 SD	0.2 SS
Logit statistic NCS	0.2 SS	0.3 SD	0.3 SS_ϵ
Raw logit NCS	0.3 SS	None	0.2 SD_ϵ
Embedding NCS	0.3 SD_ϵ	0.9 SS_ϵ	0.2 SS
Tensor FFT NCS	0.3 SS_ϵ	None	0.2 SD

It can be seen that benign-view NCS gives the most consistent results. It gives successful detection on all ten sequences at $\rho_{atk} = 0.2$ for PGD, CW, and AP. Logit statistic NCS also detects all three attacks. However, CW and AP require $\rho_{atk} = 0.3$. Raw logit NCS and Tensor FFT NCS detect PGD and AP, but not CW. Embedding NCS detects PGD and AP at $\rho_{atk} = 0.3$ and $\rho_{atk} = 0.2$, respectively, and detects CW only at $\rho_{atk} = 0.9$.

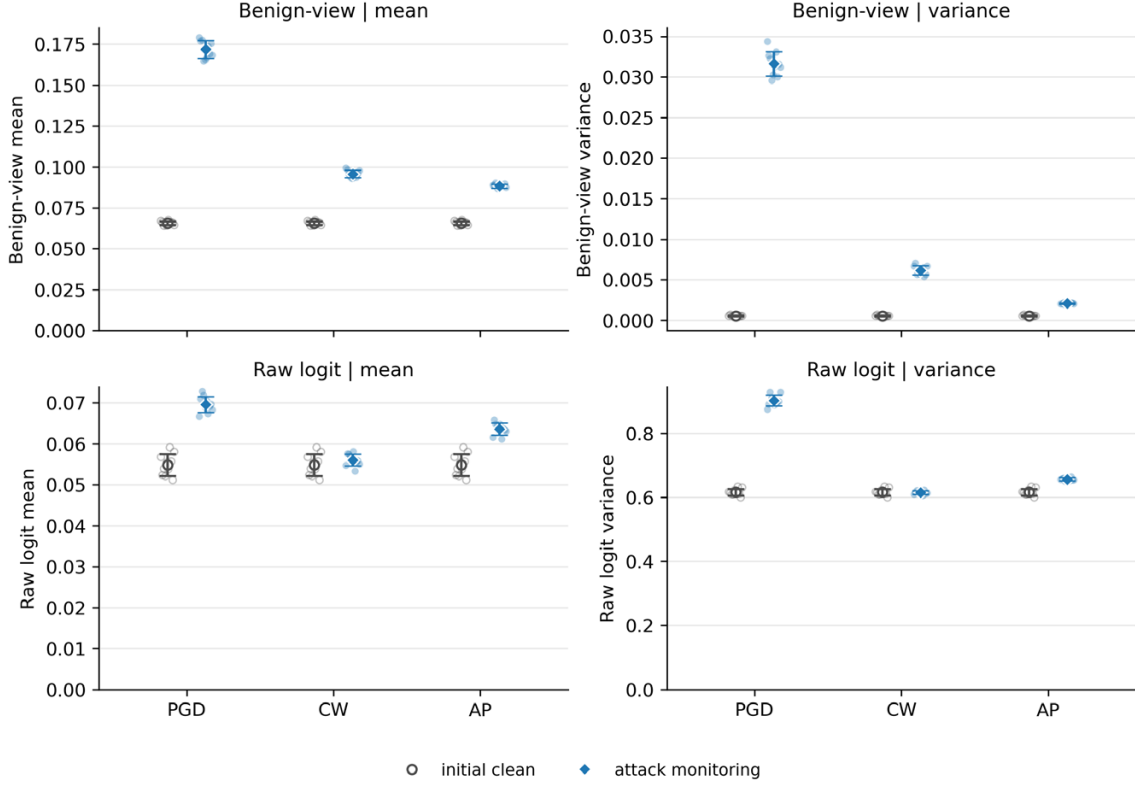


Figure 2: NCS means and variances at $\rho_{\text{atk}} = 0.3$, shown separately for PGD, CW, and AP. Gray circles denote the initial clean segments, and blue diamonds denote the monitoring segments containing AEs. Small points show the ten sampled sequences; large markers show their averages, with error bars indicating one standard deviation. For the raw logit NCS, the mean and variance are first computed for each of the 100 logit components and then averaged over components.

4.3. Analysis by NCS Score and Martingale Algorithm

We now inspect the NCS and the martingale algorithm. The goal is to explain two parts of the result. First, the NCS should expose an attack-induced mean or variance change. Second, the martingale algorithm should detect the resulting p-value sequence efficiently.

Figure 2 compares the mean and variance of the initial clean segment and the attacked monitoring segment at $\rho_{\text{atk}} = 0.3$. For the raw logit NCS, the CTM calculation still uses 100 component-wise martingales followed by pointwise averaging. The figure only summarizes the score statistics. For each sampled sequence, we compute the mean and variance for each logit component on the segment being plotted. We then average the 100 component summaries to obtain one plotted value for that sampled sequence.

The benign-view NCS shows a clear shift from the initial segment to the monitoring segment. This is visible in both mean and variance. For CW, the shift also remains clear.

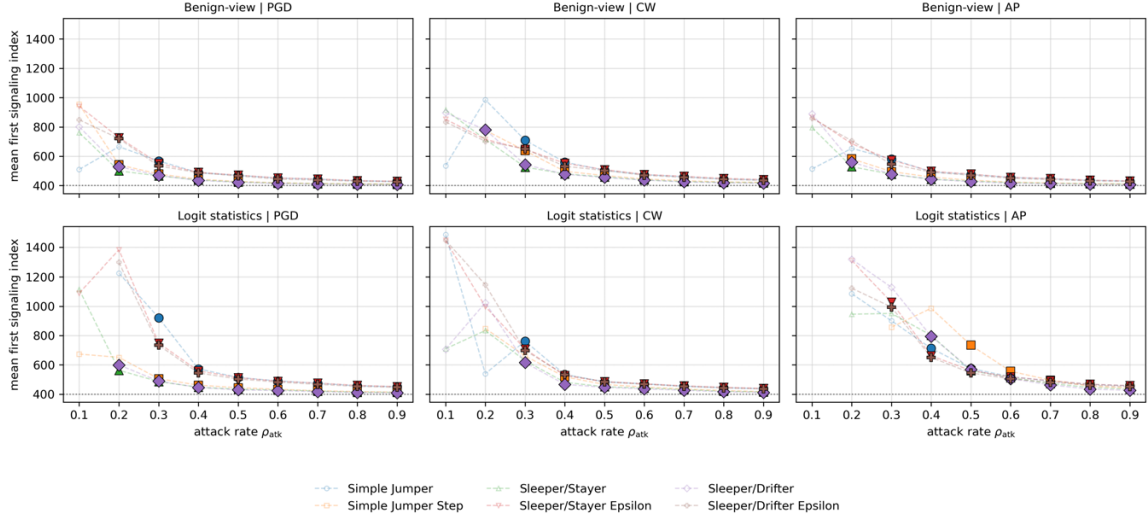


Figure 3: Mean first signaling index across ten sampled sequences for the benign-view NCS and logit statistic NCS. Filled markers indicate attack rates and algorithms with 10/10 valid detections. Pale dashed markers indicate settings that do not reach 10/10. The dotted horizontal line marks the end of the initial clean segment.

In contrast, the raw logit NCS shows weaker changes for CW. This is consistent with its failure to reach 10/10 valid detections for CW in Table 2.

This pattern is broadly compatible with the CTM analysis of Boström (2026). That study shows that the relative efficiency of CTM algorithms depends on the type of distribution shift. Epsilon betting can be effective for mean changes, whereas step betting is more effective when the shift affects the spread of the distribution. The benign-view NCS shows changes in both mean and variance, so the mean shift may also contribute to the competitive performance of algorithms with epsilon betting function, as indicated in Appendix A, Table 3. However, under our evaluation criterion, algorithms with epsilon betting function produce fewer valid detections at low attack rates than Sleeper/Stayer and Sleeper/Drifter. Consequently, the selected algorithms for the benign-view NCS are Sleeper/Drifter for PGD and CW and Sleeper/Stayer for AP, suggesting that these step based algorithms provide the most effective detection performance for this NCS in the present setting.

We next compare the first signaling index after fixing the NCS. Figure 3 shows the mean first signaling index of the benign-view NCS and the logit statistic NCS across martingale algorithms. For the benign-view NCS, the reliable algorithms become stable from $\rho_{\text{atk}} = 0.2$ for PGD, CW, and AP. At low attack rates, especially $\rho_{\text{atk}} = 0.2$ and $\rho_{\text{atk}} = 0.3$, the algorithms with epsilon-betting function remain competitive in some cases, but they do not achieve 10/10 valid detections as consistently as Sleeper/Stayer and Sleeper/Drifter. This behavior is consistent with Figure 2, where the distribution of benign-view NCS shifts in both mean and variance. The logit statistic NCS also reaches 10/10 valid detections for all three attacks, while CW and AP require $\rho_{\text{atk}} = 0.3$. These results also indicate that,

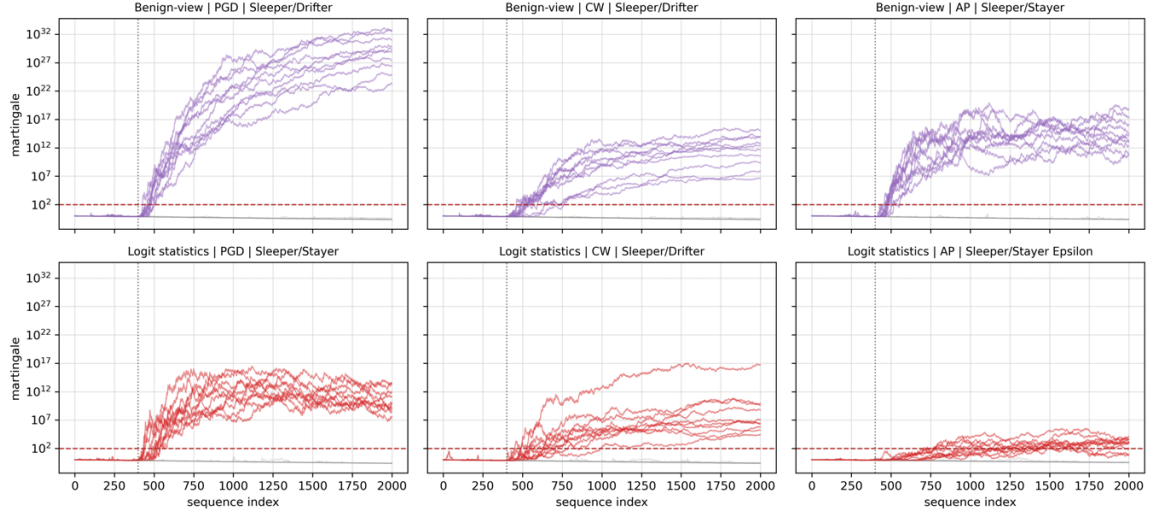


Figure 4: Martingale traces at $\rho_{\text{atk}} = 0.3$ for the benign-view NCS and logit statistic NCS with the selected algorithms in Table 2. Martingales calculated from attacked sequences are shown in color, and matched clean-control martingales are shown in gray. The horizontal dashed line is $c = 100$, and the vertical dotted line marks the end of the initial clean segment.

even among algorithms that reach 10/10 valid detections for the same NCS, the martingale algorithm still affects the first signaling index.

Figure 4 shows the corresponding martingale traces at $\rho_{\text{atk}} = 0.3$ using the selected algorithms in Table 2. At this attack rate, both NCSs give clear separation between the attack sequences and their matched clean-control sequences. The benign-view NCS crosses the threshold for all three attacks, and the logit statistic NCS also reaches the threshold consistently. However, the logit statistic martingales for AP are visibly smaller than those for PGD and CW, which suggests that, with more sampled sequences, AP detection with this NCS may be less stable than the current 10/10 result.

The two NCSs also order the three attacks differently. For the benign-view NCS, CW generally produces the smallest martingale values among the three attacks, even though AP has the smallest accuracy drop. This suggests that CW causes a smaller perturbation in the embedding space than PGD and AP, which is consistent with our previous work (Dong et al., 2025). For the logit statistic NCS, PGD and CW produce larger martingale values than AP, consistent with the larger drift these attacks induce in the ImageNet-100 logit space. This contrast follows directly from the NCS definitions: the logit statistic NCS measures changes in the logit vector, while the benign-view NCS measures embedding consistency between an image and its benign transformed view.

Finally, Figure 5 compares the benign-view NCS and the Embedding NCS on CW at $\rho_{\text{atk}} = 0.2$. The benign-view NCS reaches the threshold on all ten sampled sequences. The Embedding NCS remains close to the clean-control traces. This explains why the benign-

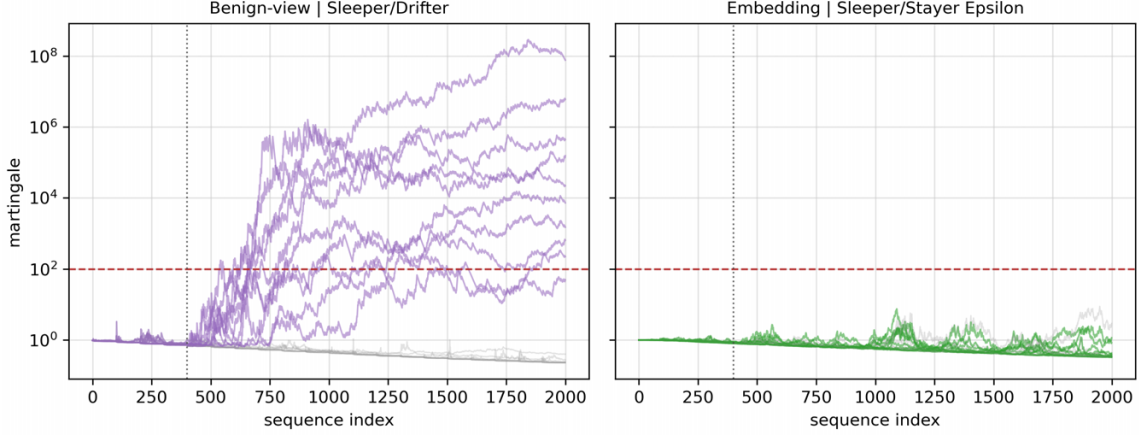


Figure 5: CW martingale traces at $\rho_{\text{atk}} = 0.2$ for the benign-view NCS and the Embedding NCS. The benign-view NCS uses the selected Sleeper/Drifter algorithm, while the Embedding NCS uses Sleeper/Stayer Epsilon, which is the algorithm reported for CW in Table 2. Martingales calculated from attacked sequences are shown in color, and matched clean-control martingales are shown in gray. The horizontal dashed line is $c = 100$, and the vertical dotted line marks the end of the initial clean segment.

view NCS detects CW at $\rho_{\text{atk}} = 0.2$, whereas the Embedding NCS only reaches 10/10 valid detections for CW at $\rho_{\text{atk}} = 0.9$.

Together, these results support the same conclusion. The NCS should first expose an attack-induced mean or variance change. Then, the martingale algorithm affects how early the CTM reaches the threshold. In this experiment, the benign-view NCS exposes the clearest distribution shift, and Sleeper/Stayer and Sleeper/Drifter achieve the most reliable signaling for this NCS.

Extended versions of the figures in this section, covering all NCSs, CTM algorithms, and attacks, are provided in Appendix B.

5. Concluding Remarks

We formulated white-box attack detection as a sequence-level distribution shift detection problem on an initial clean segment followed by a monitoring segment that may contain AEs. We evaluated five NCSs (Benign-view, Logit statistic, Raw logit, Embedding, and Tensor FFT) under three representative white-box attacks (PGD, CW, and AP) on ImageNet-100 with a pretrained ResNet-50.

Three findings emerge. (i) *Accuracy drop is not a reliable proxy for detectability*: CW reduces accuracy almost to zero yet is the hardest attack to detect, while AP with the smallest accuracy drop is detected at the same low rate as PGD by several NCSs. What matters is whether the score sequence exposes a drift the martingale can accumulate. (ii) *NCS choice matters*: the Benign-view NCS is the most consistent NCS in the present experiments, reaching 10/10 valid detections for all three attacks at $\rho_{\text{atk}} = 0.2$. However, this should not be read as a universal dominance claim: the Logit statistic NCS also detects three attacks but requires higher attack rates for CW and AP, while Raw logit, Embedding, and Tensor FFT NCSs expose only part of the attack-induced shift and fail in different ways. (iii) *NCS and algorithm for computing martingales interact*: mean-type drift is captured well by algorithms with epsilon betting, while variance-type drift favours step betting, so the best martingale algorithm depends on which type of drift a given NCS produces.

Together, these findings argue against searching for a universal NCS or universal algorithm. Instead, the results show that the effectiveness of each fixed NCS and martingale algorithm configuration depends on the distribution shift induced by the attack.

Future work. A main direction is to further investigate the theoretical properties of the averaged martingale obtained from multiple NCS components when the attacked positions are unknown. Of particular interest are possible lower bounds on the probability that the martingale reaches the threshold and a characterization of the delay from the first AE in the monitoring segment to the first signaling index. We also plan to conduct empirical validation using multiple datasets and attack methods at different attack rates.

Limitations. Several aspects of our study limit the scope of the conclusions. First, the attack methods considered here are not adaptive: they are generated against the target ResNet-50 without knowledge of the CTM detector, so they do not represent attacks that are optimized against both the model and the detection statistics. Second, each NCS is evaluated with a fixed configuration, such as the ResNet stage used for embedding extraction, the affine scale 0.84 for the benign view, and the specific FFT statistics; we did not sweep over these design choices. Third, the results are based on 10 randomly sampled sequences for each experimental configuration and are summarized by whether valid detections are achieved on all 10 sequences. This is stringent but coarse, and does not capture continuous detection rates or detection delay distributions. Finally, all experiments use ImageNet-100 with a single pretrained backbone; the behavior of the proposed NCSs on other datasets, with other architectures, and under adaptive attacks remains to be established.

Acknowledgments

This work was partially supported by the Wallenberg AI, Autonomous Systems and Software Program (WASP) funded by the Knut and Alice Wallenberg Foundation.

References

- Henrik Boström. Conformal prediction in python with crepes. In *Proc. of the 13th Symposium on Conformal and Probabilistic Prediction with Applications*, pages 236–249. PMLR, 2024.
- Henrik Boström. Testing exchangeability for multiple sequences of p-values. In *COPA*, pages 615–632, 2025.
- Henrik Boström. An investigation of conformal test martingales. In *The Importance of Being Learnable: Essays Dedicated to Alexander Gammerman*, pages 118–143. Springer, 2026.
- Anneliese Braunegg, Amartya Chakraborty, Michael Krumdick, Nicole Lape, Sara Leary, Keith Manville, Elizabeth Merkhofer, Laura Strickhart, and Matthew Walmer. Apricot: A dataset of physical adversarial attacks on object detection. In *European Conference on Computer Vision*, pages 35–50. Springer, 2020.
- Tom B Brown, Dandelion Mané, Aurko Roy, Martín Abadi, and Justin Gilmer. Adversarial patch. *arXiv preprint arXiv:1712.09665*, 2017.
- Nicholas Carlini and David Wagner. Adversarial examples are not easily detected: Bypassing ten detection methods. In *Proceedings of the 10th ACM workshop on artificial intelligence and security*, pages 3–14, 2017a.
- Nicholas Carlini and David Wagner. Towards evaluating the robustness of neural networks. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 39–57. Ieee, 2017b.
- Gavin Weiguang Ding, Kry Yik Chau Lui, Xiaomeng Jin, Luyu Wang, and Ruitong Huang. On the sensitivity of adversarial robustness to input data distributions. *ICLR (poster)*, 4, 2019.
- Genghua Dong, Roman Bresson, and Henrik Boström. Detect adversarial examples with exchangeability martingale. In *Fourteenth Symposium on Conformal and Probabilistic Prediction with Applications (COPA 2025)*, pages 758–761. PMLR, 2025.
- Javid Ebrahimi, Daniel Lowd, and Dejing Dou. On adversarial examples for character-level neural machine translation. *arXiv preprint arXiv:1806.09030*, 2018.
- Reuben Feinman, Ryan R Curtin, Saurabh Shintre, and Andrew B Gardner. Detecting adversarial samples from artifacts. *arXiv preprint arXiv:1703.00410*, 2017.
- Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.
- Sidharth Gupta, Parijat Dube, and Ashish Verma. Improving the affordability of robustness training for dnns. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition workshops*, pages 780–781, 2020.

- Paula Harder, Franz-Josef Pfreundt, Margret Keuper, and Janis Keuper. Spectraldefense: Detecting adversarial attacks on cnns in the fourier domain. In *2021 International Joint Conference on Neural Networks (IJCNN)*, pages 1–8. IEEE, 2021.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016.
- Zhiyuan He, Yijun Yang, Pin-Yu Chen, Qiang Xu, and Tsung-Yi Ho. Be your own neighborhood: Detecting adversarial example by the neighborhood relations built on self-supervised learning. *arXiv preprint arXiv:2209.00005*, 2022.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- Xingjun Ma, Bo Li, Yisen Wang, Sarah M Erfani, Sudanthi Wijewickrema, Grant Schoenebeck, Dawn Song, Michael E Houle, and James Bailey. Characterizing adversarial subspaces using local intrinsic dimensionality. *arXiv preprint arXiv:1801.02613*, 2018.
- Aleksander Madry, Aleksandar Makelov, Ludwig Schmidt, Dimitris Tsipras, and Adrian Vladu. Towards deep learning models resistant to adversarial attacks. *arXiv preprint arXiv:1706.06083*, 2017.
- Dongyu Meng and Hao Chen. Magnet: a two-pronged defense against adversarial examples. In *Proceedings of the 2017 ACM SIGSAC conference on computer and communications security*, pages 135–147, 2017.
- Jan Hendrik Metzen, Tim Genewein, Volker Fischer, and Bastian Bischoff. On detecting adversarial perturbations. *arXiv preprint arXiv:1702.04267*, 2017.
- Maria-Irina Nicolae, Mathieu Sinn, Minh Ngoc Tran, Beat Buesser, Ambrish Rawat, Martin Wistuba, Valentina Zantedeschi, Nathalie Baracaldo, Bryant Chen, Heiko Ludwig, et al. Adversarial robustness toolbox v1. 0.0. *arXiv preprint arXiv:1807.01069*, 2018.
- Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019.
- Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision (IJCV)*, 115(3):211–252, 2015. doi: 10.1007/s11263-015-0816-y.
- Yonglong Tian, Dilip Krishnan, and Phillip Isola. Contrastive multiview coding. In *Computer Vision—ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XI 16*, pages 776–794. Springer, 2020.

- Danush Kumar Venkatesh and Peter Steinbach. Detecting adversarial examples in batches—a geometrical approach. *arXiv preprint arXiv:2206.08738*, 2022.
- Vladimir Vovk. Testing randomness online. *Statistical Science*, 36(4):595–611, 2021.
- Vladimir Vovk, Alex Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World (2nd ed.)*. Springer, 2022.
- Haohan Wang, Xindi Wu, Zeyi Huang, and Eric P Xing. High-frequency component helps explain the generalization of convolutional neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8684–8694, 2020.
- Yixiang Wang, Jiqiang Liu, Xiaolin Chang, Ricardo J Rodríguez, and Jianhua Wang. Di-aa: An interpretable white-box attack for fooling deep neural networks. *Information Sciences*, 610:14–32, 2022.
- Rey Reza Wiyatno, Anqi Xu, Ousmane Dia, and Archy De Berker. Adversarial examples in modern machine learning: A review. *arXiv preprint arXiv:1911.05268*, 2019.
- Weilin Xu, David Evans, and Yanjun Qi. Feature squeezing: Detecting adversarial examples in deep neural networks. *arXiv preprint arXiv:1704.01155*, 2017.
- Dong Yin, Raphael Gontijo Lopes, Jon Shlens, Ekin Dogus Cubuk, and Justin Gilmer. A fourier perspective on model robustness in computer vision. *Advances in Neural Information Processing Systems*, 32, 2019.

Appendix A. CTM Algorithm Configurations and Detection Results

CTM algorithm configurations. The evaluated CTM algorithms differ in their investment strategy, betting function, and wake-up scheme. Simple Jumper uses taxation to redistribute capital across a fixed set of accounts and employs epsilon betting in its original form. Sleeper/Stayer gradually transfers capital from an investment account to a fixed set of accounts and originally uses fixed-threshold step betting. Sleeper/Drifter additionally activates new accounts over time and originally uses time-varying step thresholds. The three variants considered in this study retain the corresponding investment and wake-up schemes while replacing the original betting function.

Table 3: Algorithm-level CTM detection results. For each NCS and attack method, the table lists the martingale algorithm(s) that achieve the lowest attack rate with 10/10 valid detections. A valid detection means that the attack-sequence martingale reaches $c = 100$ while the matched clean-control martingale does not. Values in parentheses report the total number of valid detections over the ten sampled sequences and the nine tested attack rates, so the maximum possible value is $10 \times 9 = 90$. “None” means that no algorithm reaches 10/10 valid detections on the tested grid.

NCS family	Attack	Lowest rate	Algorithm(s) achieving the lowest rate
Raw logit NCS	PGD	0.3	Sleeper/Stayer (73); Sleeper/Drifter (71)
Raw logit NCS	CW	None	None
Raw logit NCS	AP	0.2	Sleeper/Drifter Epsilon (82); Sleeper/Drifter (81); Sleeper/Stayer (81); Sleeper/Stayer Epsilon (81); Simple Jumper Step (80)
Logit statistic NCS	PGD	0.2	Sleeper/Stayer (82); Sleeper/Drifter (80)
Logit statistic NCS	CW	0.3	Sleeper/Drifter (79); Sleeper/Drifter Epsilon (76); Sleeper/Stayer (75); Sleeper/Stayer Epsilon (75); Simple Jumper (72)
Logit statistic NCS	AP	0.3	Sleeper/Stayer Epsilon (73); Sleeper/Drifter Epsilon (72)
Embedding NCS	PGD	0.3	Sleeper/Drifter Epsilon (74); Sleeper/Stayer Epsilon (74)
Embedding NCS	CW	0.9	Sleeper/Drifter Epsilon (31); Sleeper/Stayer Epsilon (29)
Embedding NCS	AP	0.2	Sleeper/Drifter (81); Sleeper/Stayer (80); Simple Jumper Step (80)
Benign-view NCS	PGD	0.2	Sleeper/Drifter (86); Sleeper/Stayer (86); Sleeper/Stayer Epsilon (85); Sleeper/Drifter Epsilon (84); Simple Jumper Step (82)
Benign-view NCS	CW	0.2	Sleeper/Drifter (81)
Benign-view NCS	AP	0.2	Sleeper/Stayer (86); Sleeper/Drifter (85); Simple Jumper Step (80)
Tensor FFT NCS	PGD	0.3	Sleeper/Drifter Epsilon (80); Sleeper/Stayer Epsilon (80); Sleeper/Stayer (79); Sleeper/Drifter (78); Simple Jumper (75); Simple Jumper Step (74)
Tensor FFT NCS	CW	None	None
Tensor FFT NCS	AP	0.2	Sleeper/Drifter (87); Sleeper/Stayer (86); Sleeper/Drifter Epsilon (82); Sleeper/Stayer Epsilon (82); Simple Jumper Step (82); Simple Jumper (81)

Appendix B. Additional Analysis by NCS and Martingale Algorithm

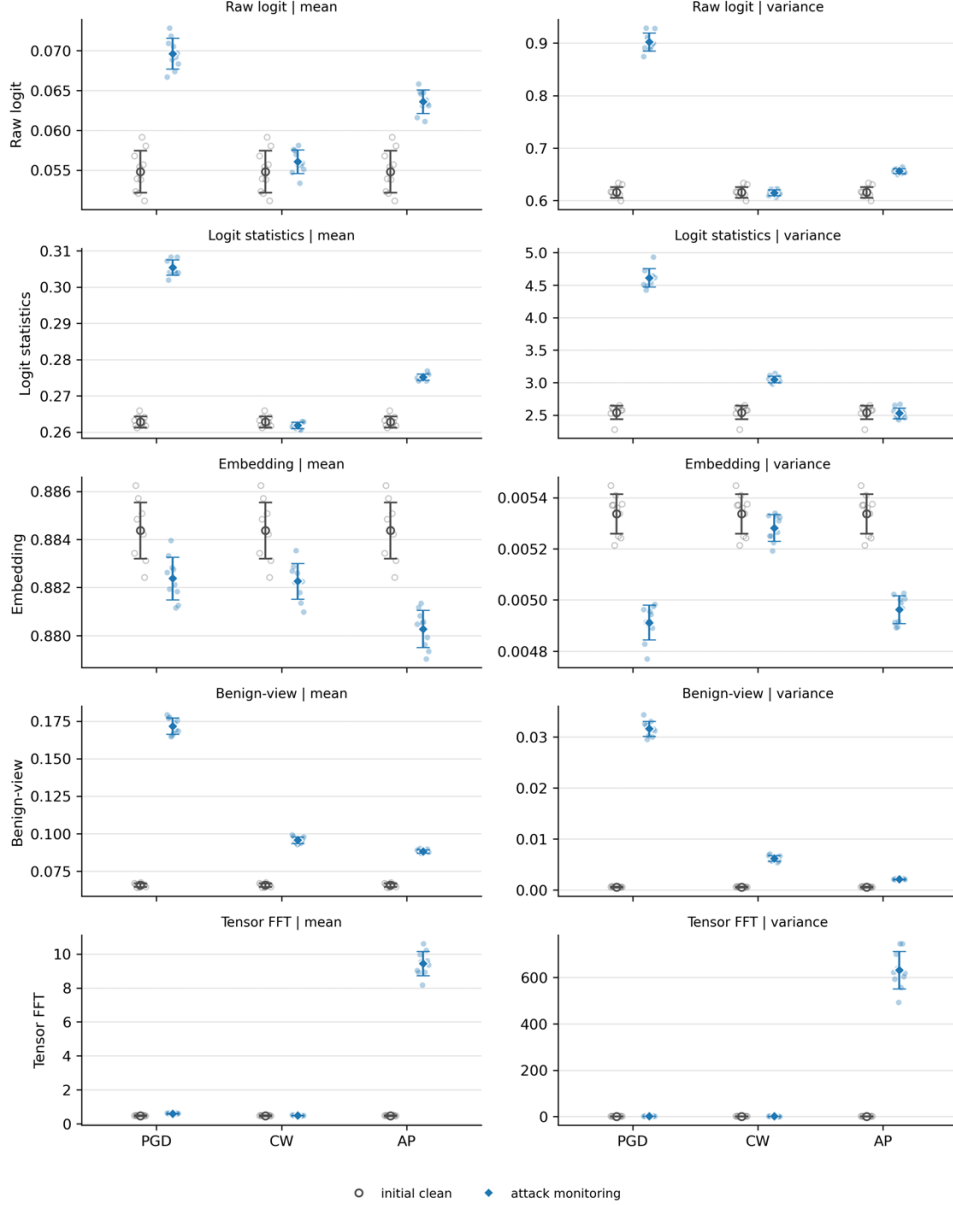


Figure 6: NCS means and variances at $\rho_{\text{atk}} = 0.3$ for all five NCSs, shown separately for PGD, CW, and AP. Gray circles denote the initial clean segments, and blue diamonds denote the monitoring segments containing AEs. Small points show the ten sampled sequences; large markers show their averages, with error bars indicating one standard deviation. For the raw logit NCS, the mean and variance are first computed for each of the 100 logit components and then averaged over components.

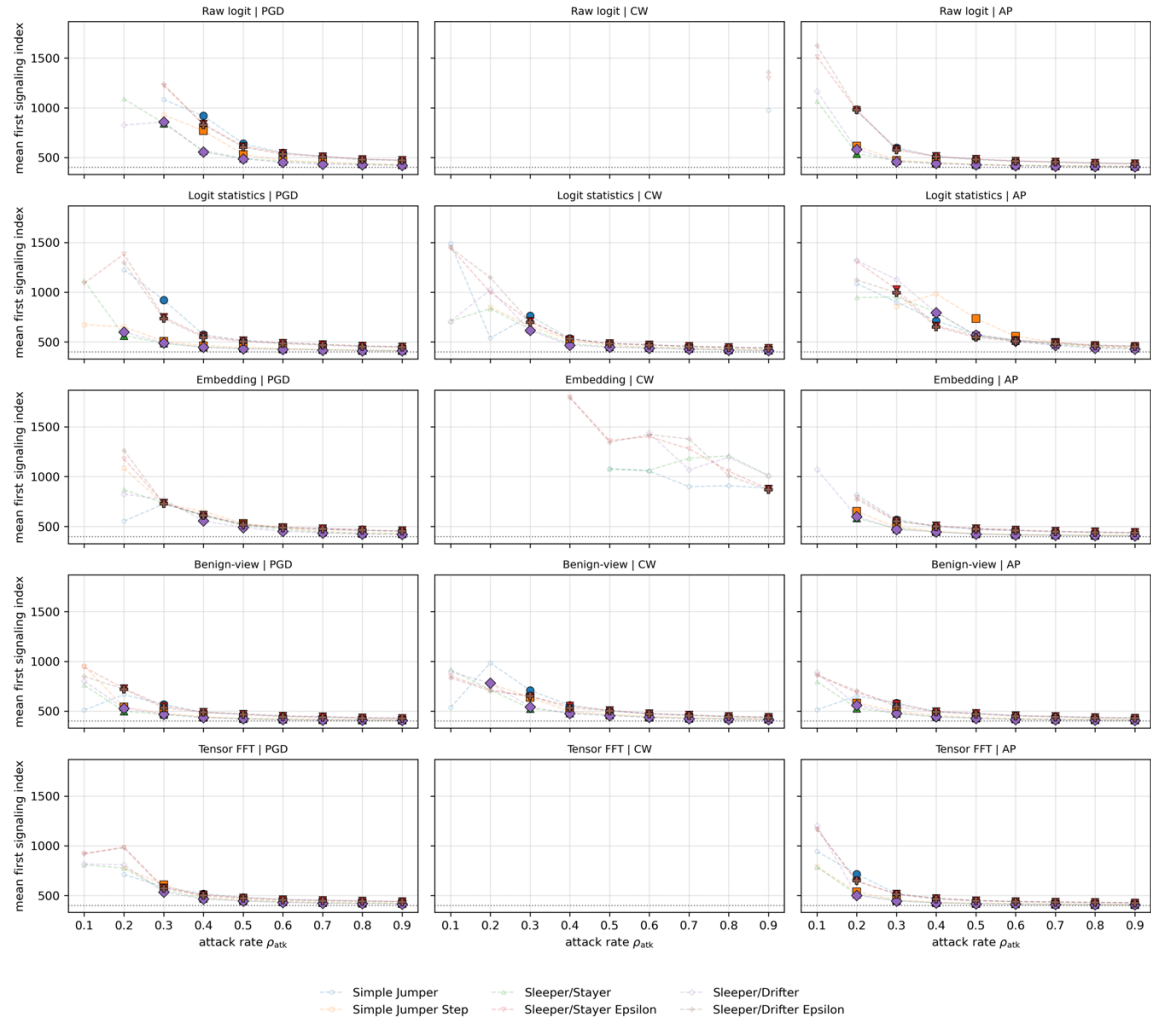


Figure 7: Mean first signaling index across ten sampled sequences for all NCS families and martingale algorithms. Filled markers indicate attack rates and algorithms with 10/10 valid detections. Pale dashed markers indicate settings that do not reach 10/10. The dotted horizontal line marks the end of the initial clean segment.

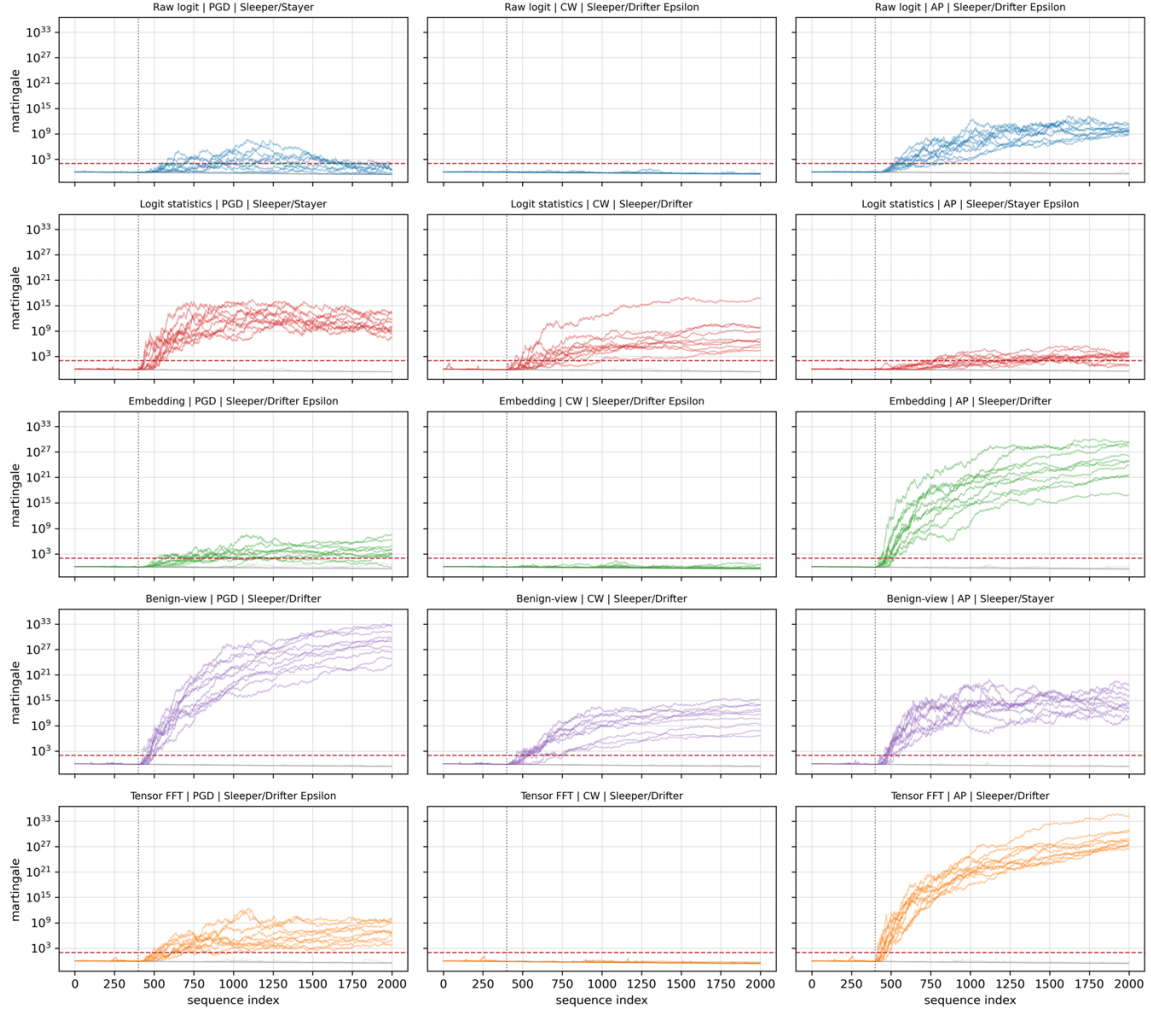


Figure 8: Martingale traces at $\rho_{\text{atk}} = 0.3$ for all NCS families with the selected martingale algorithms in Table 2. Attack-sequence martingales are shown in color, and matched clean-control martingales are shown in gray. The horizontal dashed line is $c = 100$, and the vertical dotted line marks the end of the initial clean segment.